

Deep Learning for Text Analytics



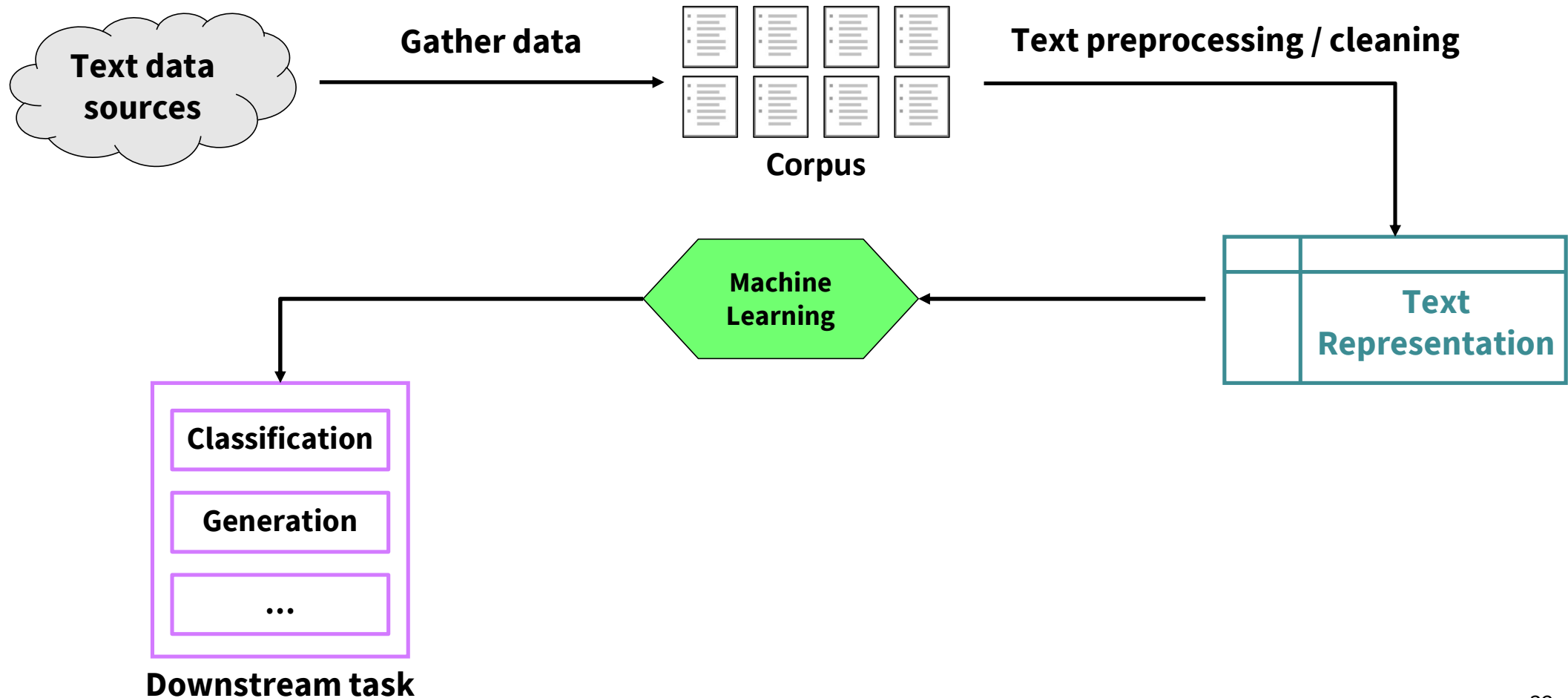
Word Embeddings

Stefan Lessmann [stefan.lessmann@hu-berlin.de]



RECAP: Natural Language Processing Pipeline

Map textual data to a numerical representation that ML methods can process

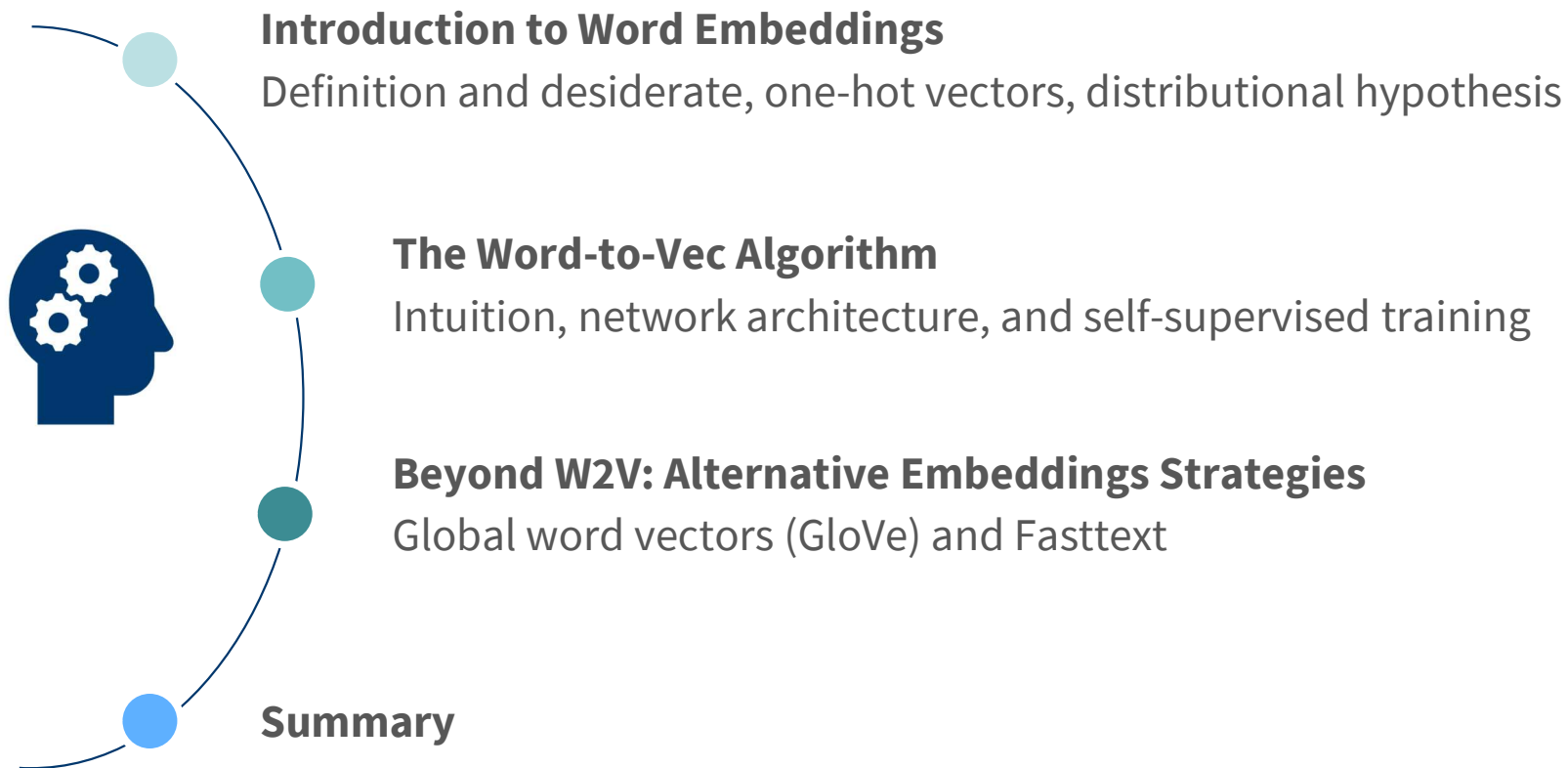


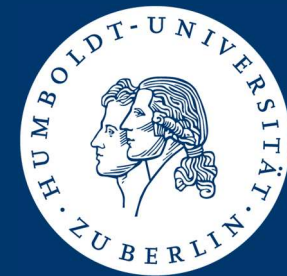
RECAP: Evolution of Natural Language Processing

Perspectives, inputs, and outputs have continuously evolved

Model Type	Input Unit	Output Unit Represents	Dimensionality	Notes
Bag of words (e.g., TF-IDF)	Document	Document	High / sparse	No order or semantics
Word Embeddings (e.g., Word2Vec)	Word	Word	Low / dense	Captures semantics
Transformers (e.g., GPT)	(short) piece of text	(short) piece of text	Low / dense	Captures semantics and context
Multimodal LLMs	Text, image, audio, etc.	Text, image, audio, etc.	Higher	Semantics and context across data types
???	???	???	???	???

Agenda



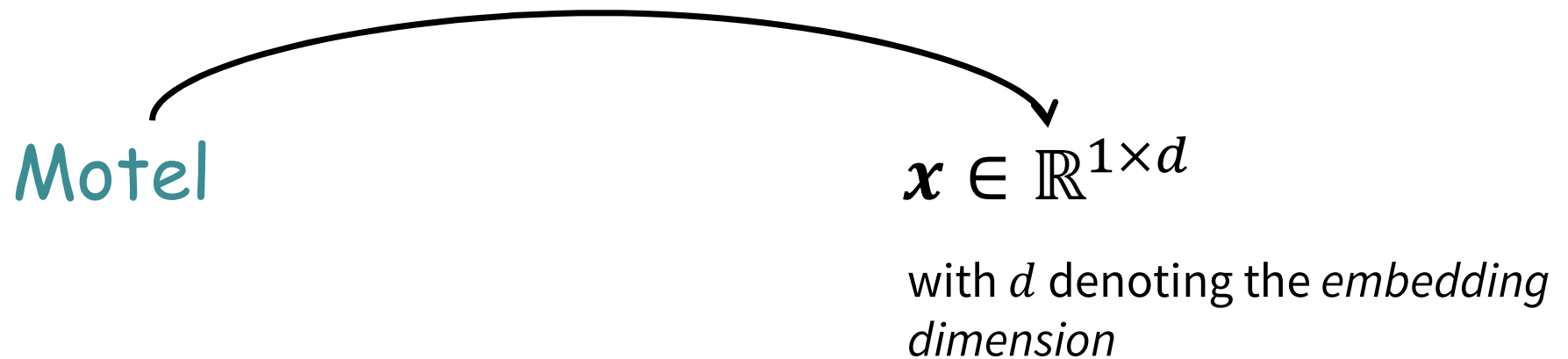


Introduction to Word Embeddings

Definition and desiderata, one-hot vectors, distributional hypothesis

Word Embeddings

An embedding represents a word (token) as a numerical vector



A First Idea: One-Hot Embeddings

Represent each word (token) as a binary vector

■ Consider a small example corpus with three sentences

Motels have parking lots

Hotels charge for car parking

Parking is expensive

■ The embedding dimension is given by the size of the vocabulary

- Here, we have ten distinct words
- Our embedding space has $d = 10$ dimensions

■ Each dimension corresponds to a binary variable “asking” if a word (token) equals a given word

- For example, you can consider X_1 a dummy variable “IsMotels?”
- Likewise, X_2 corresponds to a dummy variable “IsHave?”, X_3 is a dummy “IsParking?” and so on

	X_1	X_2	X_3	...						X_{10}
Motels	1	0	0	0	0	0	0	0	0	0
Have	0	1	0	0	0	0	0	0	0	0
Parking	0	0	1	0	0	0	0	0	0	0
Lots	0	0	0	1	0	0	0	0	0	0
Hotels	0	0	0	0	1	0	0	0	0	0
Charge	0	0	0	0	0	1	0	0	0	0
For	0	0	0	0	0	0	1	0	0	0
Car	0	0	0	0	0	0	0	1	0	0
Is	0	0	0	0	0	0	0	0	1	0
Expensive	0	0	0	0	0	0	0	0	0	1

One-Hot Embeddings Cannot Capture Words Semantics

One-hot vectors are orthogonal (by design)

■ Example of semantic relationship between words pertains to *hotel* and *motel*

- These words represent related concepts
- It is fair to say that the word *hotel* is more similar to *motel* than to other words

■ Measure similarity of embeddings

- Distance between numerical vectors
- Cosine distance is popular in NLP

Motel = [1 0 0 0 0 0 0 0 0 0 0]

Hotel = [0 0 0 0 1 0 0 0 0 0]

$$\cos \theta = \frac{\overrightarrow{\text{hotel}} \cdot \overrightarrow{\text{motel}}}{\|\overrightarrow{\text{hotel}}\| \cdot \|\overrightarrow{\text{motel}}\|} = 0$$

	X_1	X_2	X_3	...						X_{10}
Motels	1	0	0	0	0	0	0	0	0	0
Have	0	1	0	0	0	0	0	0	0	0
Parking	0	0	1	0	0	0	0	0	0	0
Lots	0	0	0	1	0	0	0	0	0	0
Hotels	0	0	0	0	1	0	0	0	0	0

Implication:

- All vectors are equally distant from another
- No notion of word similarity
- Impossible to capture semantic (or syntactic patterns)

Word Embeddings Revisited

An embedding represents a word (i.e. token) as a numerical vector

■ Desiderata

- Ensure that word vectors carry meaning and encompass the semantics of a word
- Ensure that learnt word representations can be used in downstream tasks (i.e. transfer learning)

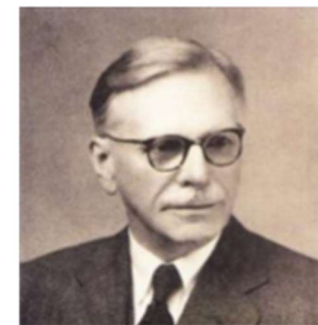
■ Distributional hypothesis

- The more semantically similar two words are, the more distributionally similar they are
- The more distributionally similar two words are, the more they will tend to occur in similar contexts

■ Modeling by means of linear algebra

- Collect distributional information in high-dimensional vectors
- Define distributional (i.e., semantic) similarity as vector similarity
- Perform dimensionality reduction
 - LSA: term co-occurrences (paradigmatic similarity)
 - LDA: context co-occurrence (topical similarity)

You shall know a word
by the company it keeps



Firth (1957)

Toward A Distributed Word Representation

Capture word semantics across latent dimensions

■ Example corpus revisited

- We can encode ten states using 4 bits
- And still have capacity for new words

■ It is no longer clear what the dimensions represent

■ Meaning of words is somehow distributed across dimensions

■ Word embedding algorithms represent words as continuous (not binary), low-dimensional (i.e., compared to vocabulary size) vectors

- Dimensions encode latent characteristics of a word
- Each dimension contributes to encoding different aspects of a word's meaning in combination with other features
- Therefore, we cannot interpret a single dimension (easily)

	X_1	X_2	X_3	X_4
Motels	0	0	0	0
Have	0	0	0	1
Parking	0	0	1	0
Lots	0	0	1	1
Hotels	0	1	0	0
Charge	0	1	0	1
For	0	1	1	0
Car	0	1	1	1
Is	1	0	0	0
expensive	1	0	0	1
?	1	0	1	0
?	1	0	1	1
?	1	1	0	0
?	1	1	0	1
?	1	1	1	0
?	1	1	1	1

A Famous Example

Capture word semantics across latent dimensions

■ Simplified embedding space with $d = 3$ dimensions

- Common choices used to range between 100 – 300 for early embeddings
- LLMs use several thousand dimensions (e.g., 7168 in DeepSeek)

■ Dimensions encode latent characteristics

■ But difference across multiple dimensions can reveal relationships

- [0.8, 0.6, 0.2] (king)
- [0.7, 0.5, 0.1] (man)
- + [0.7, 0.6, 0.2] (woman)
- = [0.8, 0.7, 0.3] = (queen)

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

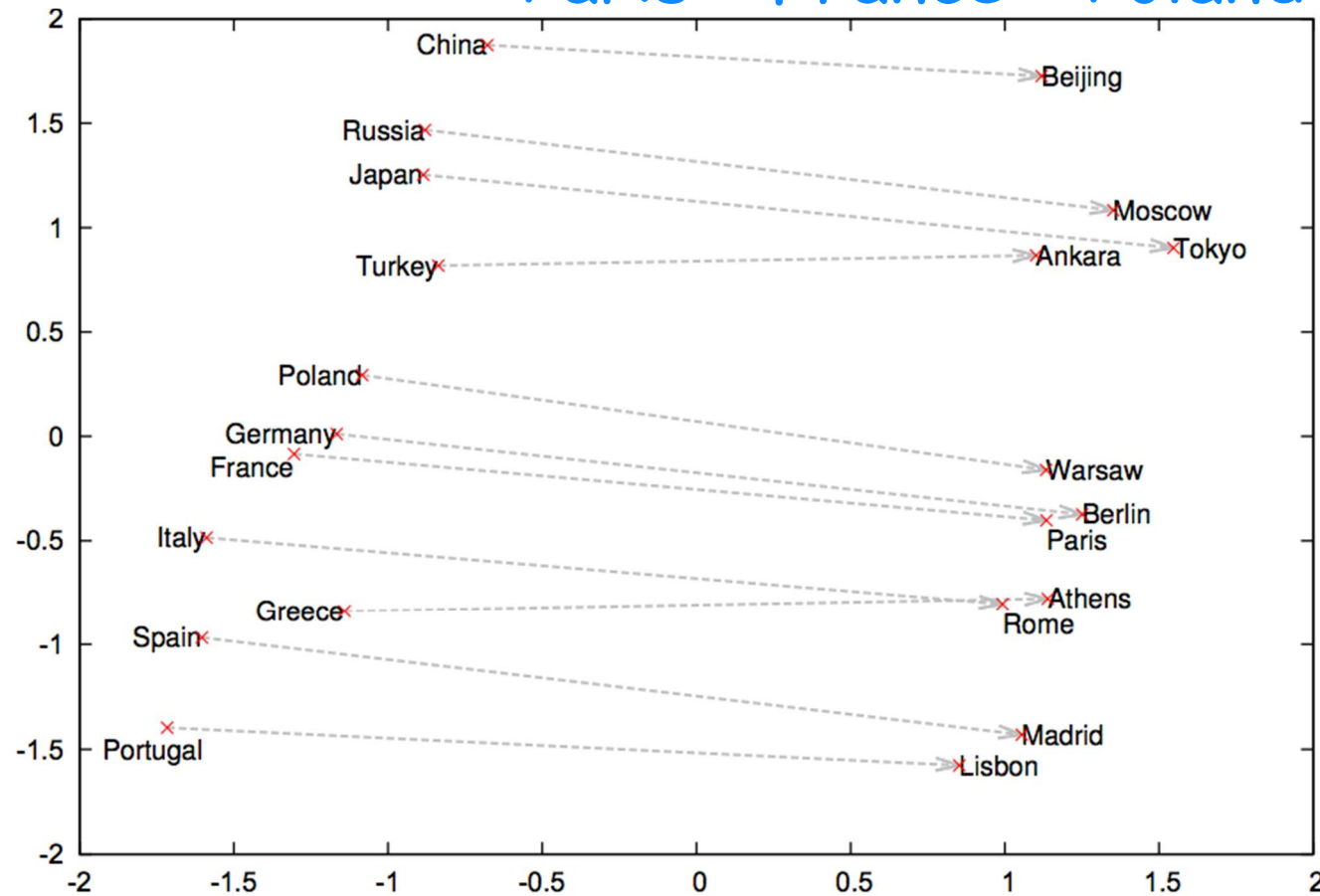
	Embedding vector		
	X_1	X_2	X_3
king	0.8	0.6	0.2
queen	0.8	0.7	0.3
man	0.7	0.5	0.1
woman	0.7	0.6	0.2

■ Semantic relationships like gender- or royalty-related patterns are *somehow* captured in the distributed representation

Examples From the W2V Paper: Countries and Capitals

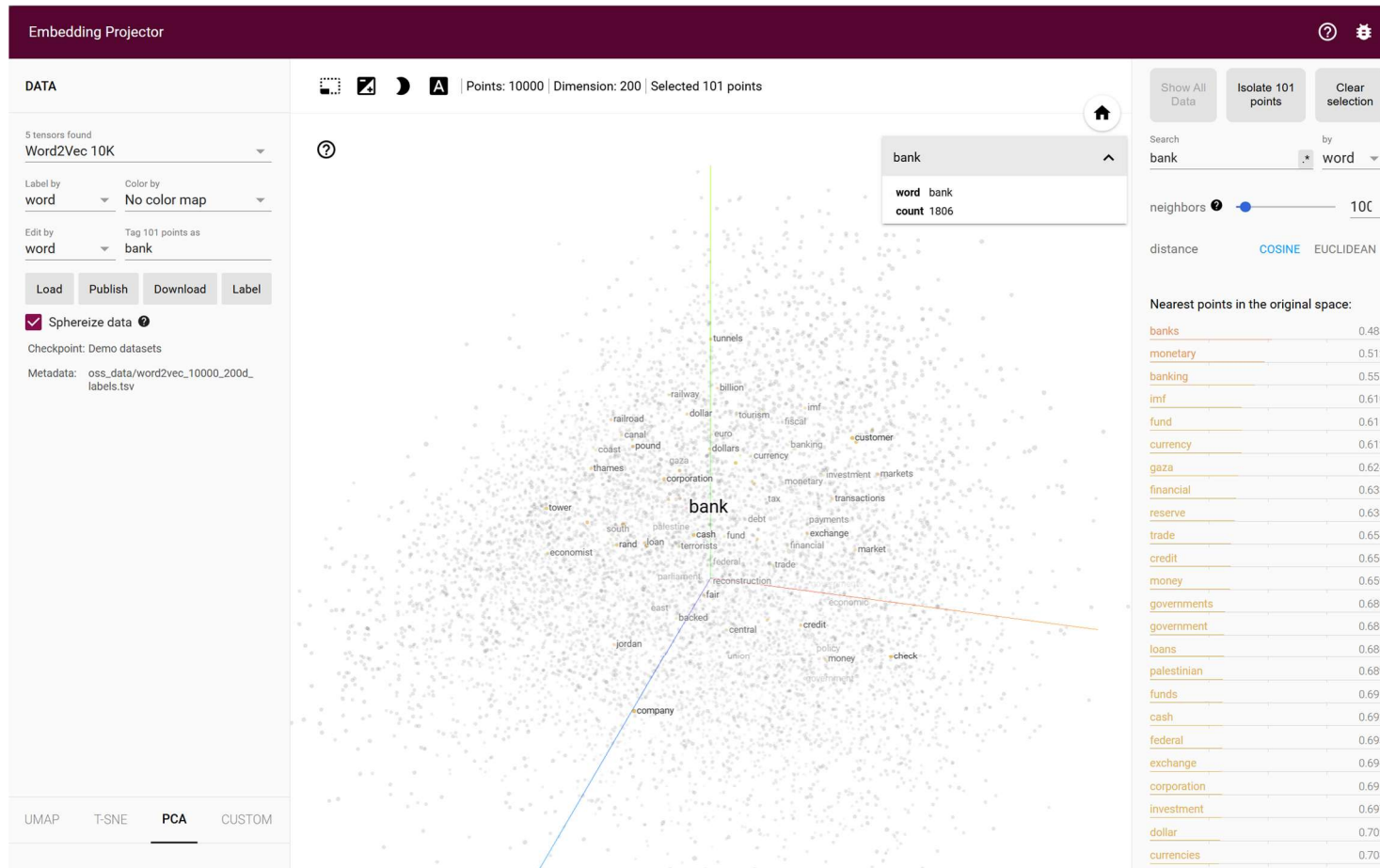
Paris - France + Poland $\approx$ Warsaw

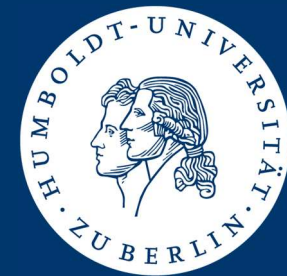
Mapping of 300-dimensional word vectors to low dimensional space for visualization



Tensorflow Embedding Projector

<https://projector.tensorflow.org/>





The Word-to-Vec Algorithm

Intuition, network architecture, and self-supervised training

The Word-to-Vec (W2V) Algorithm

Distributed Representations of Words and Phrases and their Compositionality

Tomas Mikolov
Google Inc.
Mountain View
mikolov@google.com

Ilya Sutskever
Google Inc.
Mountain View
ilyasu@google.com

Kai Chen
Google Inc.
Mountain View
kai@google.com

Greg Corrado
Google Inc.
Mountain View
gcorrado@google.com

Jeffrey Dean
Google Inc.
Mountain View
jeff@google.com

Abstract

The recently introduced continuous Skip-gram model is an efficient method for learning high-quality distributed vector representations that capture a large number of precise syntactic and semantic word relationships. In this paper we present several extensions that improve both the quality of the vectors and the training speed. By subsampling of the frequent words we obtain significant speedup and also learn more regular word representations. We also describe a simple alternative to the hierarchical softmax called negative sampling.

An inherent limitation of word representations is their indifference to word order and their inability to represent idiomatic phrases. For example, the meanings of “Canada” and “Air” cannot be easily combined to obtain “Air Canada”. Motivated by this example, we present a simple method for finding phrases in text, and show that learning good vector representations for millions of phrases is possible.

Advances in Neural Information Processing Systems 26: Proceedings of the 27th Annual Conference on Neural Information Processing Systems (NIPS 2013), Lake Tahoe, NV, USA.



Tomas Mikolov

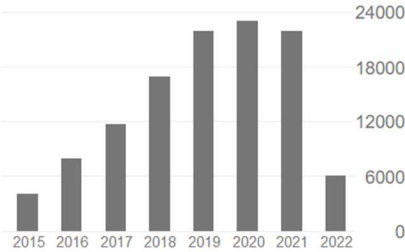
Senior Researcher, CIIRC CTU
Verified email at fb.com

Artificial Intelligence Machine Learning Language Modeling Natural Language Processing



TITLE	CITED BY	YEAR
Distributed representations of words and phrases and their compositionality T Mikolov, I Sutskever, K Chen, GS Corrado, J Dean Neural information processing systems	33397	2013
Efficient estimation of word representations in vector space T Mikolov, K Chen, G Corrado, J Dean arXiv preprint arXiv:1301.3781	28678	2013
Distributed representations of sentences and documents Q Le, T Mikolov International conference on machine learning, 1188-1196	9354	2014
Enriching word vectors with subword information P Bojanowski, E Grave, A Joulin, T Mikolov Transactions of the association for computational linguistics 5, 135-146	8005	2017
Recurrent neural network based language model. T Mikolov, M Karafiát, L Burget, J Cernocký, S Khudanpur Interspeech 2 (3), 1045-1048		

Cited by	VIEW ALL	
	All	Since 2017
Citations	117461	101844
h-index	47	46
i10-index	83	73



The Word-to-Vec (W2V) Algorithm

Learn word embeddings using a shallow feedforward neural network

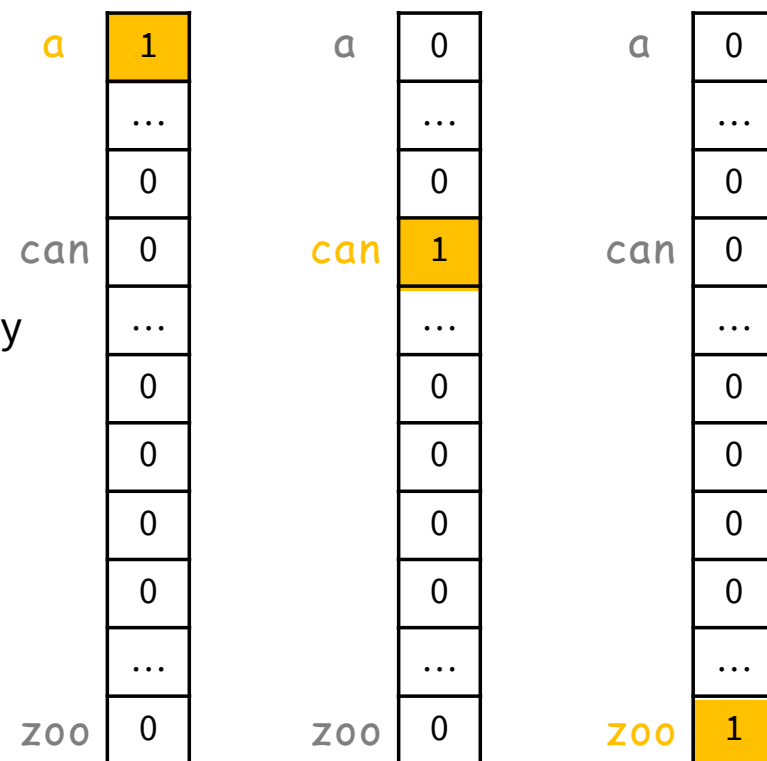
■ Core idea in W2V is to predict neighboring words

■ Recall distributional hypothesis

- The more semantically similar two words are, the more distributionally similar they are
- The more distributionally similar two words are, the more they will tend to occur in similar contexts
- Predicting neighboring words is like predicting word co-occurrences

■ W2V uses a very simple neural network

- Network input is a one-hot vector representing a word in a fixed vocabulary of size $\mathcal{V}$
- Network output is a probability distribution over words



$$\mathbf{x} \in \{0,1\}^{\mathcal{V} \times 1}$$

Self-Supervised Training

Leverage large, unlabeled corpora to learn word semantics

■ Two flavors of W2V

- Predict a **center word** from surrounding **(context) words** (continuous bag of words, CBOW)
- Predict surrounding **(context) words** given a **center word** (skip-gram)

■ Example sentence

"Even the smallest person can change the course of the future."

- Quantify surrounding words by defining a context window
- Assume we consider a context window of size 2 and take person as our central word

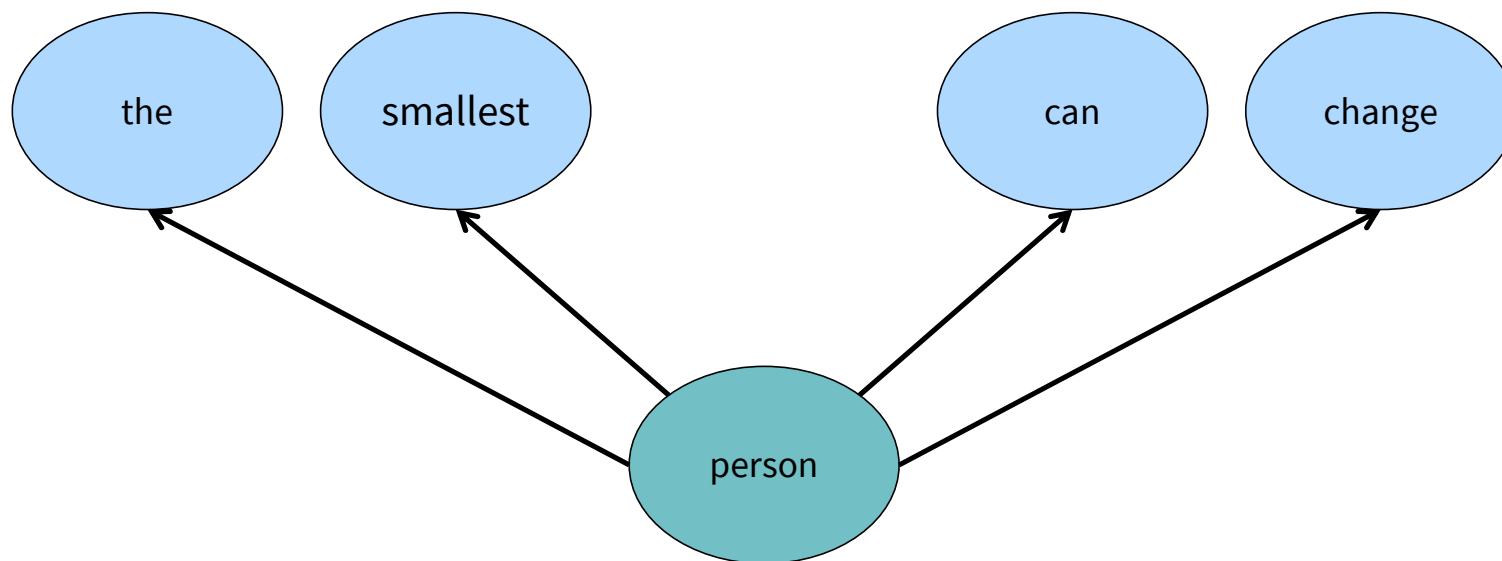
"Even **the smallest** **person** **can change** the course of the future."

The diagram shows the sentence "Even the smallest person can change the course of the future." with "person" highlighted in green and labeled "central word" above it. Brackets below "the smallest" and "can change" label them as "context words".

Self-Supervised Training Under the Skip-Gram Model

Conditional probability of generating context words given the center word

"Even the smallest person can change the course of the future."

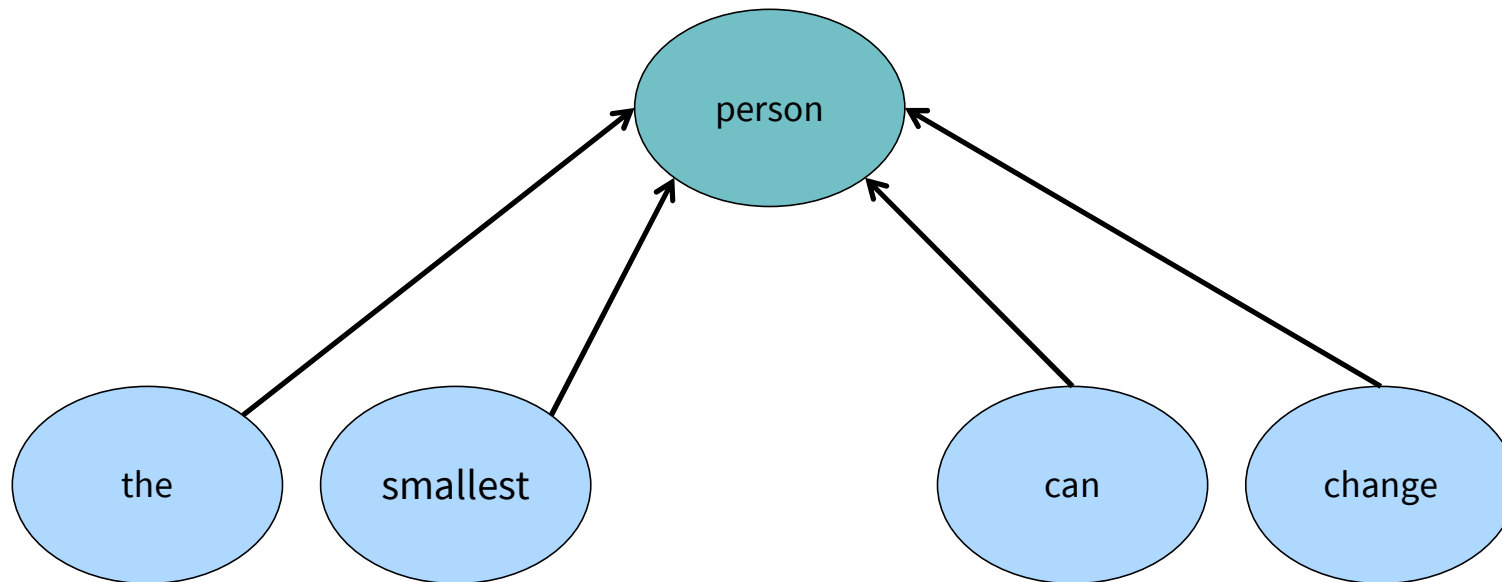


$$P(\text{the}|\text{person}) \cdot P(\text{smallest}|\text{person}) \cdot P(\text{can}|\text{person}) \cdot P(\text{change}|\text{person})$$

Self-Supervised Training Under the Continuous Bag of Word Model

Conditional probability of context words generating a center word

"Even the smallest person can change the course of the future."



$$P(\text{person} | \text{the, smallest, can, change})$$

Self-Supervised Training Under the CBOW Model

Generating training data from a (large) corpus

"Even the **smallest** person can change the course of the future."

■ Training tuples (y_i, X_i) when parsing example sentence with context size 2

□ ...

□ (**smallest**, {even, the, person, can})

$$P(x_t | x_{t-2}, x_{t-1}, x_{t+1}, x_{t+2})$$

Self-Supervised Training Under the CBOW Model

Generating training data from a (large) corpus

"Even the smallest person can change the course of the future."

■ Training tuples (y_i, X_i) when parsing example sentence with context size 2

- ...
- (smallest, {even, the, person, can})
- (person, {the, smallest, can, change})

$$P(x_t | x_{t-2}, x_{t-1}, x_{t+1}, x_{t+2})$$

Self-Supervised Training Under the CBOW Model

Generating training data from a (large) corpus

"Even the smallest person can change the course of the future."

■ Training tuples (y_i, X_i) when parsing example sentence with context size 2

- ...
- (smallest, {even, the, person, can})
- (person, {the, smallest, can, change})
- (can, {smallest, person, change, the})

$$P(x_t | x_{t-2}, x_{t-1}, x_{t+1}, x_{t+2})$$

Self-Supervised Training Under the CBOW Model

Generating training data from a (large) corpus

"Even the smallest person can change the course of the future."

■ Training tuples (y_i, X_i) when parsing example sentence with context size 2

- ...
- (smallest, {even, the, person, can})
- (person, {the, smallest, can, change})
- (can, {smallest, person, change, the})
- (change, {smallest, can, the, course})

$$P(x_t | x_{t-2}, x_{t-1}, x_{t+1}, x_{t+2})$$

Self-Supervised Training Under the CBOW Model

Generating training data from a (large) corpus

"Even the smallest person can change the course of the future."

■ Training tuples (y_i, X_i) when parsing example sentence with context size 2

- ...
- (smallest, {even, the, person, can})
- (person, {the, smallest, can, change})
- (can, {smallest, person, change, the})
- (change, {smallest, can, the, course})
- (the, {can, change, course, of})

$$P(x_t | x_{t-2}, x_{t-1}, x_{t+1}, x_{t+2})$$

Self-Supervised Training Under the CBOW Model

Generating training data from a (large) corpus

"Even the smallest person can change the course of the future."

■ Training tuples (y_i, X_i) when parsing example sentence with context size 2

- ...
- (smallest, {even, the, person, can})
- (person, {the, smallest, can, change})
- (can, {smallest, person, change, the})
- (change, {smallest, can, the, course})
- (the, {can, change, course, of})
- (course, {change, the, of, the})

$$P(x_t | x_{t-2}, x_{t-1}, x_{t+1}, x_{t+2})$$

Self-Supervised Training Under the CBOW Model

Generating training data from a (large) corpus

"Even the smallest person can change the course of the future."

■ Training tuples (y_i, X_i) when parsing example sentence with context size 2

- ...
- (smallest, {even, the, person, can})
- (person, {the, smallest, can, change})
- (can, {smallest, person, change, the})
- (change, {smallest, can, the, course})
- (the, {can, change, course, of})
- (course, {change, the, of, the})
- (of, {the, course, the, future})
- ...

$$P(x_t | x_{t-2}, x_{t-1}, x_{t+1}, x_{t+2})$$

Self-Supervised Training in Word2Vec

■ Setting up a classification task by parsing a (large) corpus

■ Skip-Gram model

- Conditional probability of a center word generating surrounding context words
- Formally

$$P(x_{t-2}|x_t) \cdot P(x_{t-1}|x_t) \cdot P(x_{t+1}|x_t) \cdot P(x_{t+2}|x_t)$$

■ Continuous bag of words model

- Conditional probability of context words generating a center word
- Formally

$$P(x_t | x_{t-2}, x_{t-1}, x_{t+1}, x_{t+2})$$

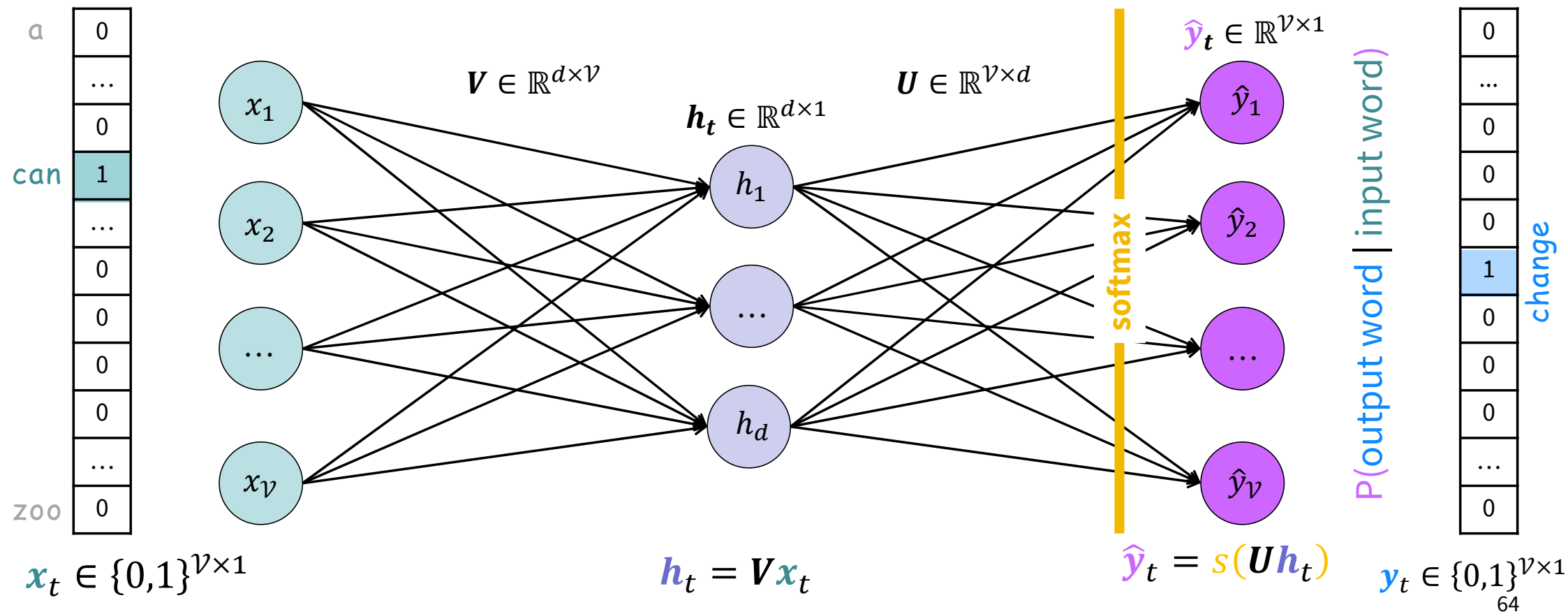
■ Size of the context window is a meta-parameter

■ No need for target labels or text annotation → self-supervision

Setting up the Neural Network

Shallow NN with hidden layer size d and two weight matrices V and U

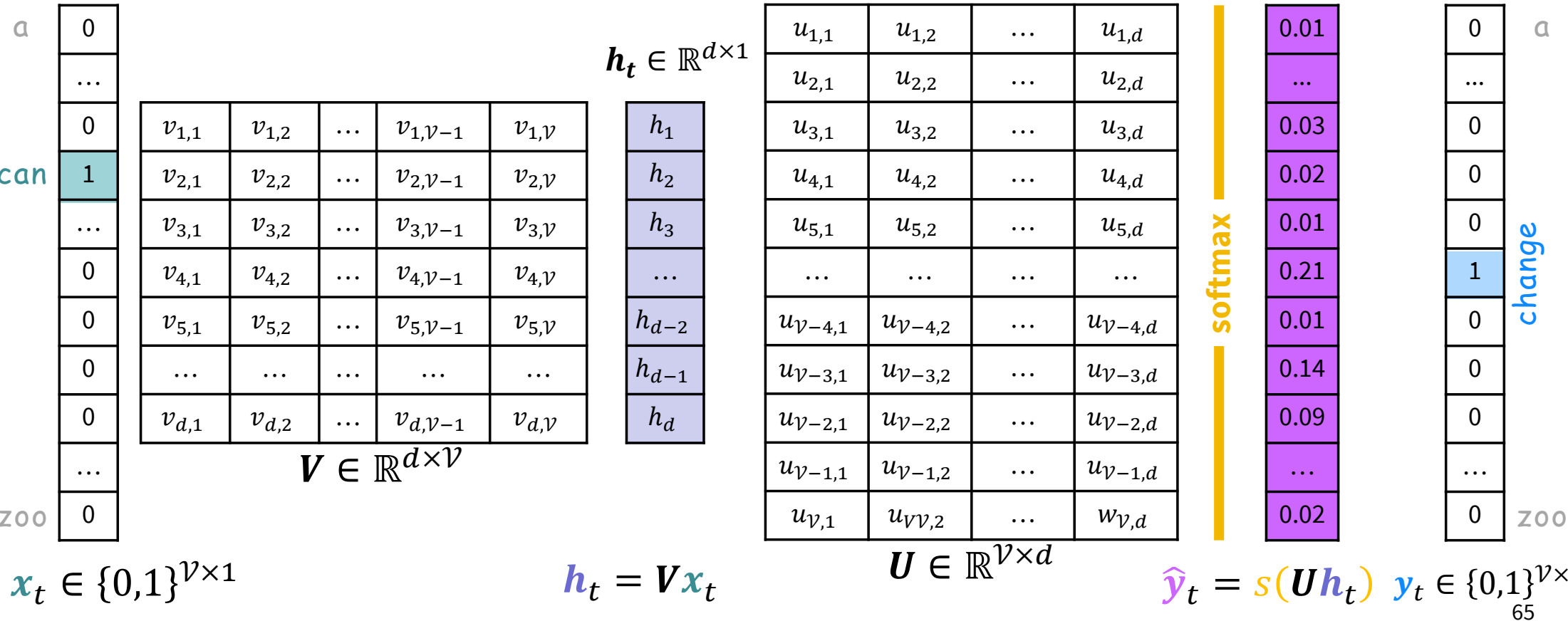
"Even the smallest person **can change** the course of the future."



Setting up the Neural Network

Shallow NN with hidden layer size d and two weight matrices V and U

“Even the smallest person can change the course of the future.”



Setting up the Neural Network

Dot product with one-hot-vector is like indexing

"Even the smallest person can change the course of the future."

$v_{1,1}$	$v_{1,2}$	$\dots$	$v_{1,\nu-1}$	$v_{1,\nu}$
$v_{2,1}$	$v_{2,2}$	$\dots$	$v_{2,\nu-1}$	$v_{2,\nu}$
$v_{3,1}$	$v_{3,2}$	$\dots$	$v_{3,\nu-1}$	$v_{3,\nu}$
$v_{4,1}$	$v_{4,2}$	$\dots$	$v_{4,\nu-1}$	$v_{4,\nu}$
$v_{5,1}$	$v_{5,2}$	$\dots$	$v_{5,\nu-1}$	$v_{5,\nu}$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
$v_{d,1}$	$v_{d,2}$	$\dots$	$v_{d,\nu-1}$	$v_{d,\nu}$

$$V \in \mathbb{R}^{d \times \nu}$$

x_1
x_2
$\dots$
$x_{\nu-1}$
x_ν

$$\mathbf{x}_t \in \{0,1\}^{\nu \times 1}$$

$h_1 = v_{1,1}x_1 + v_{1,2}x_2 + \dots + v_{1,\nu-1}x_{\nu-1} + v_{1,\nu}x_\nu$
$h_2 = v_{2,1}x_1 + v_{2,2}x_2 + \dots + v_{2,\nu-1}x_{\nu-1} + v_{2,\nu}x_\nu$
$\dots$
$h_{d-1} = v_{d-1,1}x_1 + v_{d-1,2}x_2 + \dots + v_{d-1,\nu-1}x_{\nu-1} + v_{d-1,\nu}x_\nu$
$h_d = v_{d,1}x_1 + v_{d,2}x_2 + \dots + v_{d,\nu-1}x_{\nu-1} + v_{d,\nu}x_\nu$

$$\mathbf{h}_t \in \mathbb{R}^{d \times 1}$$

$$\mathbf{h}_t = V \mathbf{x}_t$$

Setting up the Neural Network

Dot product with one-hot-vector is like indexing

"Even the smallest person can change the course of the future."

$v_{1,1}$	$v_{1,2}$	$\dots$	$v_{1,\nu-1}$	$v_{1,\nu}$
$v_{2,1}$	$v_{2,2}$	$\dots$	$v_{2,\nu-1}$	$v_{2,\nu}$
$v_{3,1}$	$v_{3,2}$	$\dots$	$v_{3,\nu-1}$	$v_{3,\nu}$
$v_{4,1}$	$v_{4,2}$	$\dots$	$v_{4,\nu-1}$	$v_{4,\nu}$
$v_{5,1}$	$v_{5,2}$	$\dots$	$v_{5,\nu-1}$	$v_{5,\nu}$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
$v_{d,1}$	$v_{d,2}$	$\dots$	$v_{d,\nu-1}$	$v_{d,\nu}$

$$V \in \mathbb{R}^{d \times \nu}$$

$x_1 = 0$
$x_2 = 1$
$\dots$
$x_{\nu-1} = 0$
$x_\nu = 0$

$$\mathbf{x}_t \in \{0,1\}^{\nu \times 1}$$

$h_1 = 0 + v_{1,2}x_2 + \dots + 0$
$h_2 = 0 + v_{2,2}x_2 + \dots + 0$
$\dots$
$h_{d-1} = 0 + v_{d-1,2}x_2 + \dots + 0$
$h_d = 0 + v_{d,2}x_2 + \dots + 0$

$$\mathbf{h}_t \in \mathbb{R}^{d \times 1}$$

$$\mathbf{h}_t = V \mathbf{x}_t$$

Word embeddings

After training, weight matrix V gives embeddings of words in the vocabulary

"Even the smallest person can change the course of the future."

a	0
	...
	0
can	1
	...
	0
	0
	0
	0
	...
zoo	0

$$\mathbf{x}_t \in \{0,1\}^{\mathcal{V} \times 1}$$

a			can				zoo
$v_{1,1}$	...	$v_{1,t-1}$	$v_{1,t}$	$v_{1,t+1}$	...	$v_{1,\mathcal{V}-1}$	$v_{1,\mathcal{V}}$
$v_{2,1}$	...	$v_{2,t-1}$	$v_{2,t}$	$v_{2,t+1}$	...	$v_{2,\mathcal{V}-1}$	$v_{2,\mathcal{V}}$
$v_{3,1}$	...	$v_{3,t-1}$	$v_{3,t}$	$v_{3,t+1}$	...	$v_{3,\mathcal{V}-1}$	$v_{3,\mathcal{V}}$
$v_{4,1}$	...	$v_{4,t-1}$	$v_{4,t}$	$v_{4,t+1}$	...	$v_{4,\mathcal{V}-1}$	$v_{4,\mathcal{V}}$
$v_{5,1}$	...	$v_{5,t-1}$	$v_{5,t}$	$v_{5,t+1}$	...	$v_{5,\mathcal{V}-1}$	$v_{5,\mathcal{V}}$
...	...	...	...	...	...	...	...
$v_{d,1}$	...	$v_{d,t-1}$	$v_{d,t}$	$v_{d,t+1}$	...	$v_{d,\mathcal{V}-1}$	$v_{d,\mathcal{V}}$

$$\mathbf{V} \in \mathbb{R}^{d \times \mathcal{V}}$$

Word vector of can of embedding dimension d

$$\mathbf{h}_t \in \mathbb{R}^{d \times 1}$$

h_1
h_2
h_3
...
h_{d-2}
h_{d-1}
h_d

$$\mathbf{h}_t = \mathbf{V} \mathbf{x}_t$$

Neural Network Training under Skip-Gram Model

"Even the smallest person can change the course of the future."



Context window of size 2

■ Assume conditional independence of context words given center word

$$P(\text{smallest, person, change, the} | \text{can})$$

$$= P(\text{smallest} | \text{can}) \cdot P(\text{person} | \text{can}) \cdot P(\text{change} | \text{can}) \cdot P(\text{the} | \text{can})$$

■ Model conditional probability of observing a context word given center word by softmax operation:

$$P(w_o | w_c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \mathbf{v}_c)}$$

With context word w_o and center word w_c
and corresponding embedding vectors $\mathbf{u}_o, \mathbf{v}_c \in \mathbb{R}^d$

Word2Vec Adjustments

Network training is infeasible on larger corpora

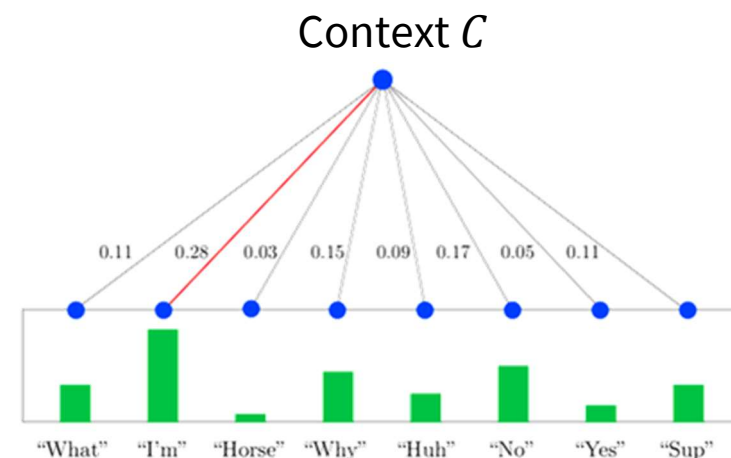
■ Large corpus essential to learn word meaning

■ High-dimensional softmax is the bottleneck

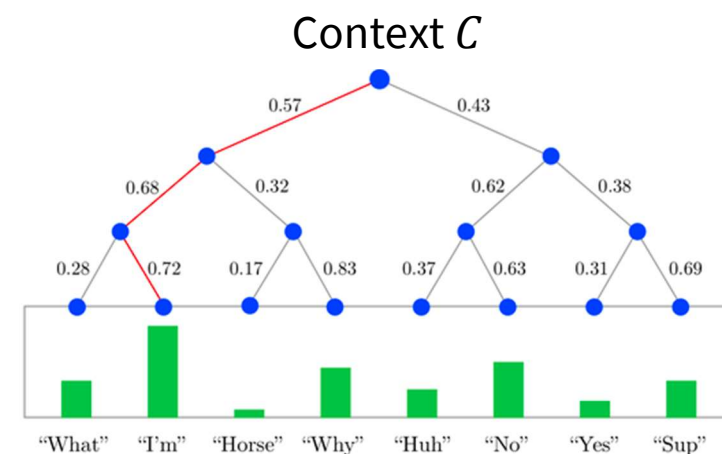
- $P(w_o | w_c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \mathbf{v}_c)}$
- Gradient update requires the conditional probabilities of all words in the vocabulary $\mathcal{V}$
- Size of the vocabulary is very large

■ Hierarchical softmax

- Accelerate computations by calculating conditional probabilities as multi-layer binary tree
- Proposed by Morin & Bengio (2005)



$$P(w_o | w_c)$$



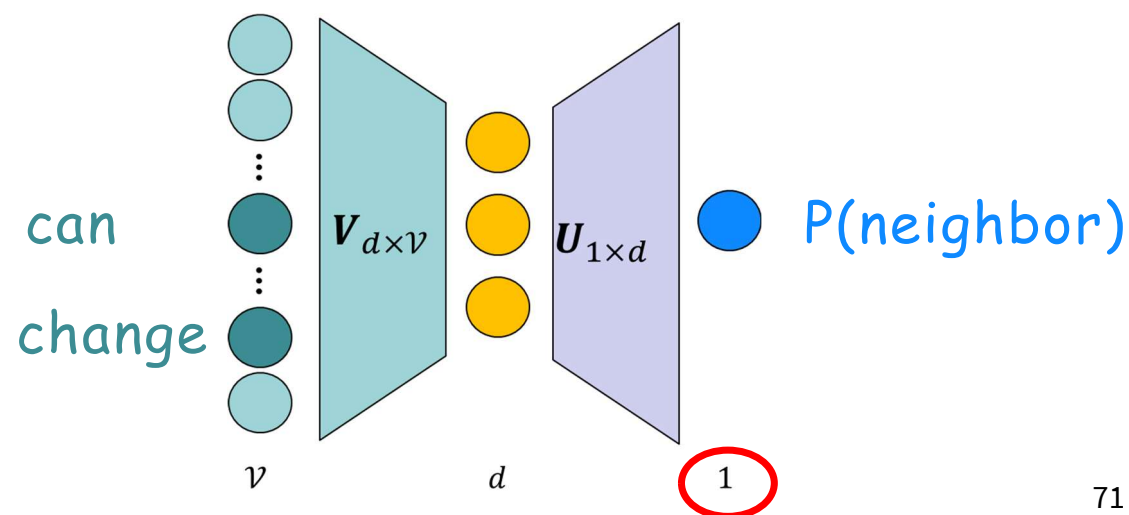
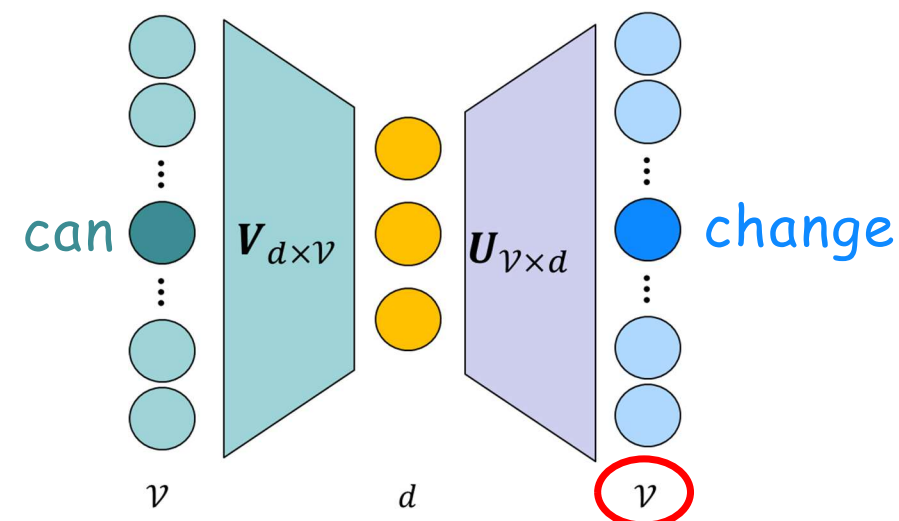
Word2Vec Adjustments

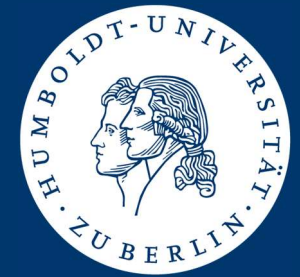
Network training is infeasible on larger corpora

■ Negative sampling

- Original W2K task: predict conditional probability of neighboring words
- Change task to binary classification: predict if two words are neighbors

"Even the smallest person can change the course of the future."





Beyond W2V: Alternative Embeddings Strategies

Global word vectors (GloVe) and Fasttext

FastText

Exploit morphology information through subword embedding

■ Words have an internal structure, which word2vec does not take into account

- Words appear in different forms in different contexts, where forms change with suffixes
- Examples : **dog** & **dogs** is analog to **cat** & **cats**; **boy** & **boyfriend** is analog to **girl** & **girlfriend**, ...
- No recognition of internal word structure in word2vec

■ Fasttext solution: move from word to character-level embedding

- Each word is represented as a bag of character n-grams
- Example for word **matter** with $n = 3$: **<ma, mat, att, tte, ter, er>**
- Better suited for rare words and able to handle new words not part of the vocabulary

■ Target word vector in skip-gram model

- Where $\mathbf{z}_g$ is the vector of subword g in the dictionary
- And the training of (sub)word vectors is the same as in word2vec

$$\mathbf{v}_c = \sum_{g \in G_w} \mathbf{z}_g$$

GloVe: Global Vectors for Word Representation

Pennington et al. (2014)

■ Objectives

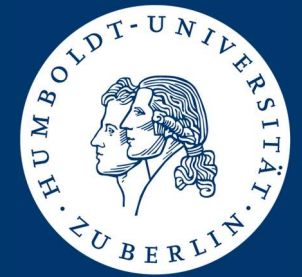
- Create word vectors that capture meaning in a vector space (like word-to-vec)
- Take advantage of global count statistics instead of only local information

■ GloVe and W2V are mathematically very similar

- W2V creates training data by streaming sentences
- GloVe learns based on a word co-occurrence matrix
- GloVe word vectors predict **co-occurrence ratios**

■ Main difference is the loss function for

- W2V: cross-entropy loss over normalized probabilities
- GloVe: log mean square error between predicted and observed **un-normalized probabilities**



Summary

Summary



Learning goals

- NLP foundations and bag-of-word model
- Word embeddings and their training



Findings

- High-dimensional, sparse count-based representations ignore word order and context
- Word embeddings
 - Represent words as low dim. dense vectors
 - Capture semantics and meaning of words
 - Self-supervised training using W2V or others



What next

- Tutorial on NLP foundations
- Recurrent neural networks for sequential data

Credits & Disclaimer

- **Materials have been inspired by and draw upon numerous great resources available online**
- **The authors acknowledge the use of and wish to express their gratitude to the following sources**
 - Jay Alammar: The Illustrated Word2vec, <http://jalammar.github.io/illustrated-word2vec/>
 - Keita Kurita (2018), Paper Dissected: “Glove: Global Vectors for Word Representation” Explained, <http://mlexplained.com/2018/04/29/paper-dissected-glove-global-vectors-for-word-representation-explained/>
 - Nishan Subedi (2018): FastText under the hood, <https://towardsdatascience.com/fasttext-under-the-hood-11efc57b2b3>
 - A Coyler: “The amazing power of word vectors”, <https://blog.acolyer.org/2016/04/21/the-amazing-power-of-word-vectors/>
 - Zhang et al.: Dive into Deep Learning (<http://d2l.ai/index.html>)
 - Stanford University: Natural Language Processing with Deep Learning (<https://web.stanford.edu/class/cs224n/>)
 - Oxford University: Deep Natural Language Processing course (<https://github.com/oxford-cs-deepnlp-2017>)

Thank you for your attention!

Stefan Lessmann

Chair of Information Systems
School of Business and Economics
Humboldt-University of Berlin, Germany

Tel. +49.30.2093.5742

Fax. +49.30.2093.5741

stefan.lessmann@hu-berlin.de

<http://bit.ly/hu-wi>

www.hu-berlin.de

